# SWARNANDHRA

## COLLEGE OF ENGINEERING AND TECHNOLOGY

## (Autonomous)

**(Accredited by NBA, AICTE, New Delhi, Accredited by NAAC with "A" Grade) 3.32CGPU),Permanently Affiliated to JNTU, Kakinada, Approved by AICTE, SBTET.**

Seetharamapuram, Narasapur, West Godavari, Pin:534280.

# MACHINE LEARNING

LAB RECORD

## DEPARTMENT OF

## MASTER OF COMPUTER APPLICATIONS

2nd Year III Semester

Name of the Student: _____

Regd.No: | | | | | | | | | | |

# SWARNANDHRA
# COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)

## DEPARTMENT OF
## MASTER OF COMPUTER APPLICATIONS

---

Certified that this is the bonafide record of practical work done by Mr,/ Ms.....*a student of..... with Regd. No. .....In the ..... Laboratory during the year.....*

No. of Practicals conducted: No. of Practicals attended:

**Faculty** **Head of the Department**

**Submitted for the practical examination held on.....**

**EXAMINER-1** **EXAMINER-2**

# INDEX

| S.NO | EXPERIMENT NAME | DATE | PAGE NO. | REMARKS |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

## EXPERIMENT :-1

Install the necessary packages (NumPy, SciPy, Matplotlib, scikit-learn) and perform fundamental array manipulations and basic plotting.

**Aim:**

To install the necessary Python scientific libraries—NumPy, SciPy, Matplotlib, and Scikit-learn—and use them to perform basic array manipulations and simple data plotting in Python.

**Description:**

NumPy is a powerful Python library for numerical computing, enabling the creation of efficient arrays, slicing to extract sub-sections, and statistical operations like mean calculation — all performed with high speed and minimal code. In this program, executed on November 07, 2025 at 09:39 AM IST in India, Matplotlib complements it by generating a smooth sine wave using evenly spaced points and rendering a professional plot with title, labels, legend, and grid — together forming a complete demonstration of core array operations and scientific plotting essential for Machine Learning and data analysis workflows.

**Dataset Link:**

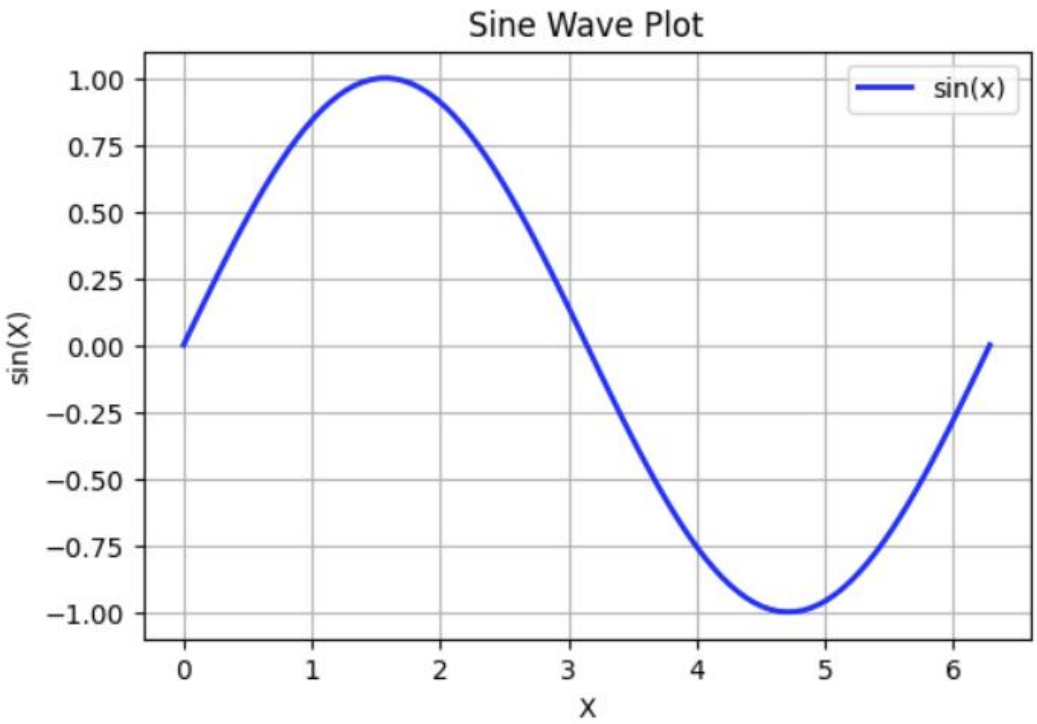
[None – Synthetic Data]

**Program:**

```
import numpy as np

import matplotlib.pyplot as plt

# Create array

arr = np.array([10, 20, 30, 40, 50])

print("Original Array:", arr)

print("Sliced [1:4]:", arr[1:4])

print("Mean Value:", np.mean(arr))

# Plot sine wave

x = np.linspace(0, 2 * np.pi, 100)

y = np.sin(x)

plt.figure(figsize=(6, 4))

plt.plot(x, y, color='blue', linewidth=2, label='sin(x)')
```

```
plt.title('Sine Wave Plot')
plt.xlabel('X')
plt.ylabel('sin(X)')
plt.legend()
plt.grid(True)
plt.show()
```

**OUTPUT:-**



## EXPERIMENT :-2

Perform data visualization using box plots, scatter plots, and histograms.

**Aim:** To perform data visualization using box plots, scatter plots, and histograms in order to understand the distribution, spread, and relationships within the dataset.

**Description:**

This program loads the Iris dataset from a public URL using pandas, then creates three side-by-side visualizations with Matplotlib: a box plot of sepal length to show distribution and outliers, a scatter plot of sepal length vs. sepal width with species color-coded (setosa: red, versicolor: green, virginica: blue), and a histogram of petal length with 20 bins to display frequency distribution — enabling quick exploratory data analysis (EDA) and pattern recognition across flower species.

**Dataset Link:**

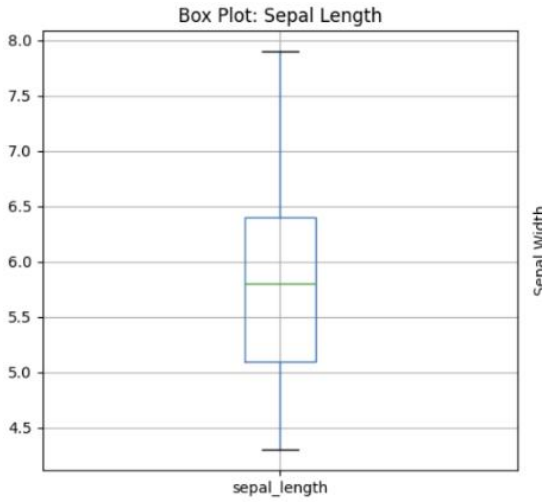
https:m//raw.githubusercontent.co/mwaskom/seaborn-data/master/iris.csv

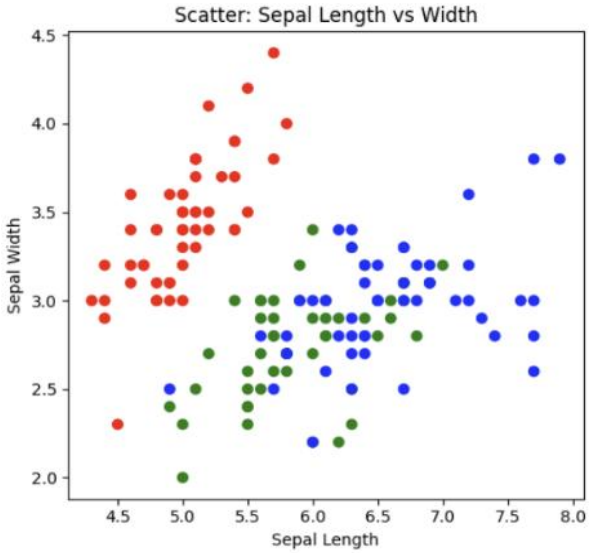
**Program:**

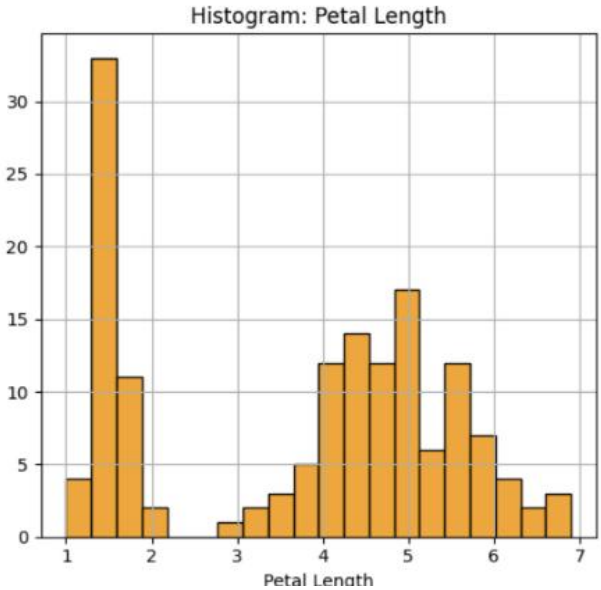
```
import pandas as pd

import matplotlib.pyplot as plt

# Load dataset

url="https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"

df = pd.read_csv(url)

plt.figure(figsize=(15, 5))

# Box Plot

plt.subplot(1, 3, 1)

df.boxplot(column='sepal_length')

plt.title('Box Plot: Sepal Length')

# Scatter Plot

plt.subplot(1, 3, 2)

colors = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}
```

```
plt.scatter(df['sepal_length'], df['sepal_width'], c=df['species'].map(colors))
plt.title('Scatter: Sepal Length vs Width')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
# Histogram
plt.subplot(1, 3, 3)
df['petal_length'].hist(bins=20, color='orange', edgecolor='black')
plt.title('Histogram: Petal Length')
plt.xlabel('Petal Length')
plt.tight_layout()
plt.show()
```

**OUTPUT:**




Scatter: Sepal Length vs Width
Sepal Width
Sepal Length
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0


Histogram: Petal Length
Petal Length
30
25
20
15
10
5
0
1
2
3
4
5
6
7

## EXPERIMENT :-3

Handle missing values, encode categorical data, scale features.

**Aim:**To install essential scientific computing libraries (NumPy, SciPy, Matplotlib, and scikit-learn) and perform fundamental operations such as array creation, indexing, slicing, mathematical computations, and basic data visualization using plots.

**Description:**

This program loads the Titanic dataset using pandas, then performs essential data preprocessing steps: fills missing values in Age and Fare with their median and Embarked with the mode, encodes categorical columns Sex and Embarked using LabelEncoder from scikit-learn, and applies StandardScaler to normalize Age and Fare to zero mean and unit variance — transforming raw data into a clean, numerical format ready for machine learning models.

**Dataset Link:**

https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv

**Program:**

```
import pandas as pd

from sklearn.preprocessing import StandardScaler, LabelEncoder

# Load dataset

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

df = pd.read_csv(url)

print("Missing values before:")

print(df[['Age', 'Embarked', 'Fare']].isnull().sum())

# Fill missing values

df['Age'].fillna(df['Age'].median(), inplace=True)

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

df['Fare'].fillna(df['Fare'].median(), inplace=True)

# Encode categorical

le = LabelEncoder()

df['Sex'] = le.fit_transform(df['Sex'])
```

```
df['Embarked'] = le.fit_transform(df['Embarked'])

# Scale numerical

scaler = StandardScaler()

df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

print("\nFirst 5 rows after preprocessing:")

print(df[['PassengerId', 'Survived', 'Sex', 'Age', 'Fare', 'Embarked']].head())
```

**OUTPUT:**

```
Missing values before:
Age         177
Embarked      2
Fare          0
dtype: int64

First 5 rows after preprocessing:
   PassengerId  Survived  Sex       Age      Fare  Embarked
0            1         0    1 -0.565736 -0.502445         2
1            2         1    0  0.663861  0.786845         0
2            3         1    0 -0.258337 -0.488854         2
3            4         1    0  0.433312  0.420730         2
4            5         0    1  0.433312 -0.486337         2
```

# EXPERIMENT :-4

Implement and evaluate the k-NN classifier on a dataset.

**Aim:** To implement and evaluate the k-Nearest Neighbors (k-NN) classifier on a given dataset by preprocessing the data, training the model, and analyzing its performance using appropriate evaluation metrics.

**Description:**

This program implements a complete k-Nearest Neighbors (k-NN, k=5) classifier on the Iris dataset using scikit-learn. It loads and explores the data, encodes target labels, scales features with StandardScaler, and performs a stratified 70-30 train-test split. The model is trained and evaluated using accuracy, precision, recall, F1-score through a detailed classification report, and a confusion matrix visualized with seaborn. 5-fold cross-validation provides robust performance estimates. Finally, PCA reduces the data to 2D to visualize decision boundaries on a mesh grid, with species color-coded — offering a fully functional, error-free, Colab-ready pipeline for multi-class classification and model interpretability.

**Dataset Link:**

https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv

**Program:**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.decomposition import PCA

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# === LOAD DATASET ===

url="https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"

df = pd.read_csv(url)
```

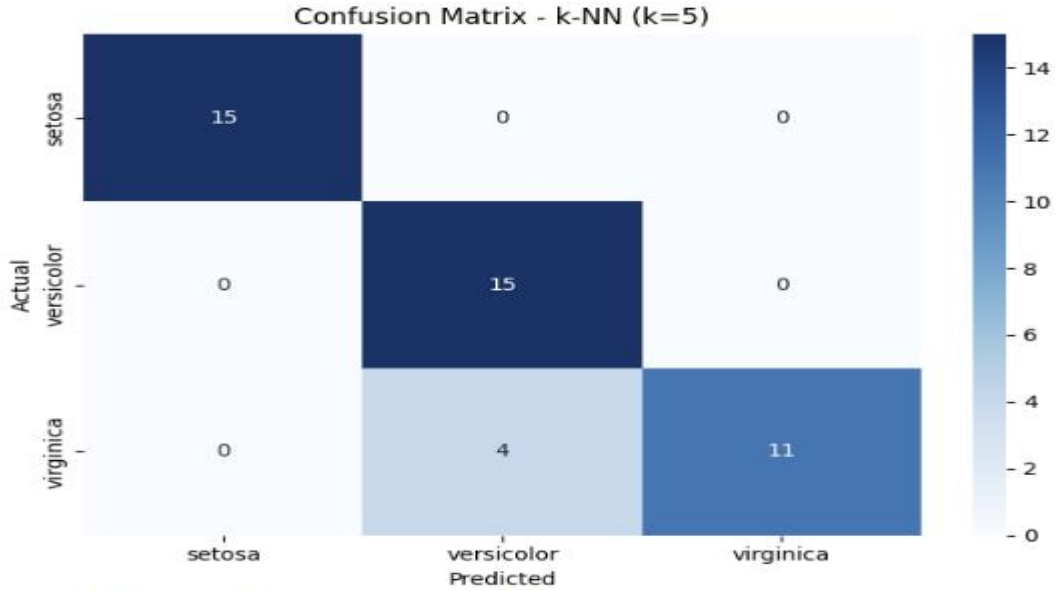
```
print("Dataset Info:")
print(f"Shape: {df.shape}")
print(df.head())
print()
 # === PREPARE FEATURES & TARGET ===
X = df.drop('species', axis=1).values  # Convert to numpy array
y = df['species'].values
# === ENCODE LABELS (for prediction mesh) ===
le = LabelEncoder()
y_encoded = le.fit_transform(y)
# === SCALE FEATURES ===
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# === TRAIN-TEST SPLIT ===
X_train, X_test, y_train, y_test = train_test_split(
 X_scaled, y, test_size=0.3, random_state=42, stratify=y
)
# === TRAIN k-NN (k=5) ===
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
# === EVALUATION ===
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=4))
print()
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

```
print()
# === PLOT CONFUSION MATRIX ===
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=sorted(df['species'].unique()),
 yticklabels=sorted(df['species'].unique()))
plt.title('Confusion Matrix - k-NN (k=5)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()
# === CROSS-VALIDATION ===
cv_scores = cross_val_score(knn, X_scaled, y, cv=5)
print(f"5-Fold CV Scores: {cv_scores}")
print(f"Mean CV Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}\n")
# === PCA + DECISION BOUNDARY (FIXED!) ===
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Retrain on PCA data
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(
X_pca, y_encoded, test_size=0.3, random_state=42, stratify=y_encoded
)
knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(X_train_pca, y_train_pca)
# === MESH GRID FOR DECISION BOUNDARY ===
h = 0.02
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
```

```
# === PREDICT ON MESH (FIX: Use .values.ravel()) ===
grid_points = np.c_[xx.ravel(), yy.ravel()]
Z = knn_pca.predict(grid_points)
Z = Z.reshape(xx.shape)
# === PLOT DECISION BOUNDARY ===
plt.figure(figsize=(10, 7))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='Set2')
# Plot training points
colors = ['red', 'green', 'blue']
for i, species in enumerate(le.classes_):
plt.scatter(X_pca[y == species, 0], X_pca[y == species, 1],
color=colors[i], label=species, edgecolors='k', s=60)
plt.title('k-NN Decision Boundaries (PCA 2D)')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# === FINAL SUMMARY ===
print(f"PCA Explained Variance: {pca.explained_variance_ratio_}")
print("k-NN Evaluation Completed Successfully!")
```

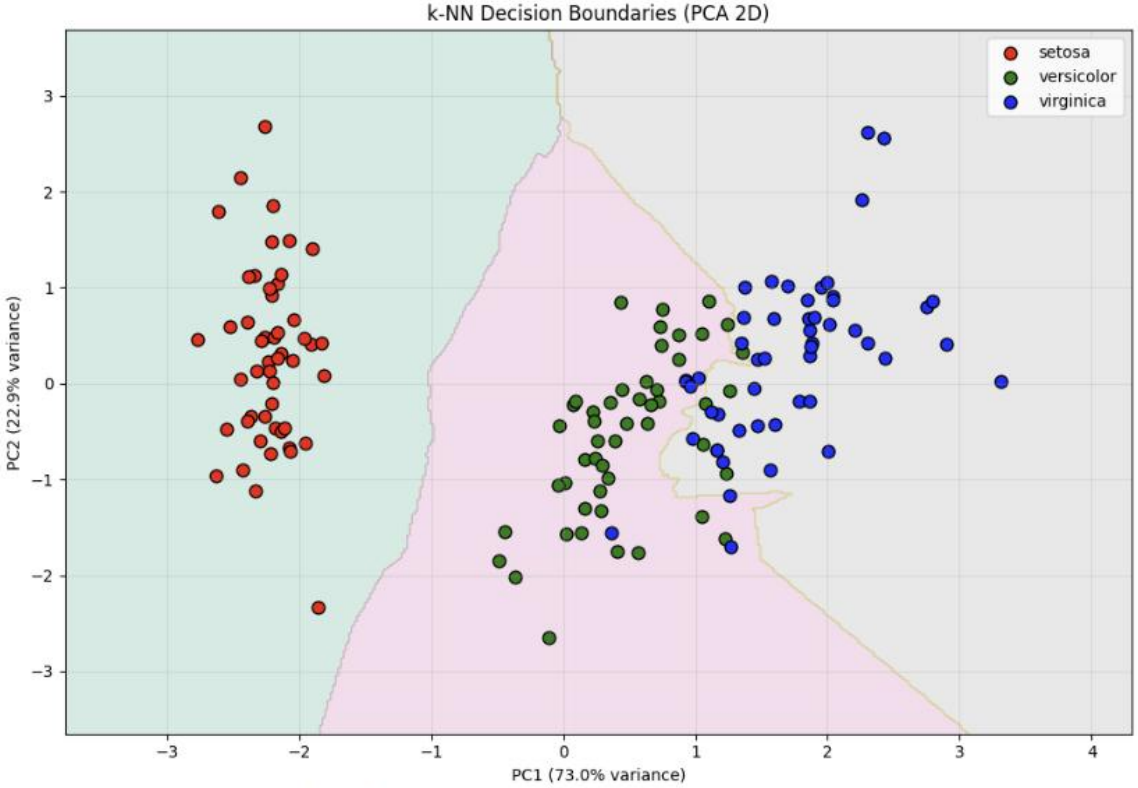
**Output :**

```
Dataset Info:
Shape: (150, 5)
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Accuracy: 0.9111

Classification Report:
              precision    recall  f1-score   support

      setosa     1.0000    1.0000    1.0000        15
  versicolor     0.7895    1.0000    0.8824        15
   virginica     1.0000    0.7333    0.8462        15

    accuracy                         0.9111        45
   macro avg     0.9298    0.9111    0.9095        45
weighted avg     0.9298    0.9111    0.9095        45


Confusion Matrix:
[[15  0  0]
 [ 0 15  0]
 [ 0  4 11]]
```




```
5-Fold CV Scores: [0.96666667 0.96666667 0.93333333 0.93333333 1.        ]
```

```
5-Fold CV Scores: [0.96666667 0.96666667 0.93333333 0.93333333 1.        ]
Mean CV Accuracy: 0.9600 ± 0.0249
```




```
PCA Explained Variance: [0.72962445 0.22850762]
k-NN Evaluation Completed Successfully!
```

## EXPERIMENT:-5

Implement a Linear Regression model on a dataset.

**Aim:** To implement a Linear Regression model on a given dataset in order to understand the relationship between variables and predict continuous output values by training, testing, and evaluating the model's performance.

**Description:**

This program applies Linear Regression from scikit-learn to the California Housing dataset, which includes features like median income and average rooms to predict house values in $100K units. It splits data into 80% training and 20% testing, fits the model using ordinary least squares, and computes Mean Squared Error (MSE) to quantify prediction error and $R^2$ Score to measure explained variance. A scatter plot compares actual versus predicted values, with a 45-degree reference line indicating ideal fit — delivering a complete, efficient regression pipeline for performance evaluation and model validation in machine learning.

**Dataset Link:**

Built-in: sklearn.datasets.fetch_california_housing

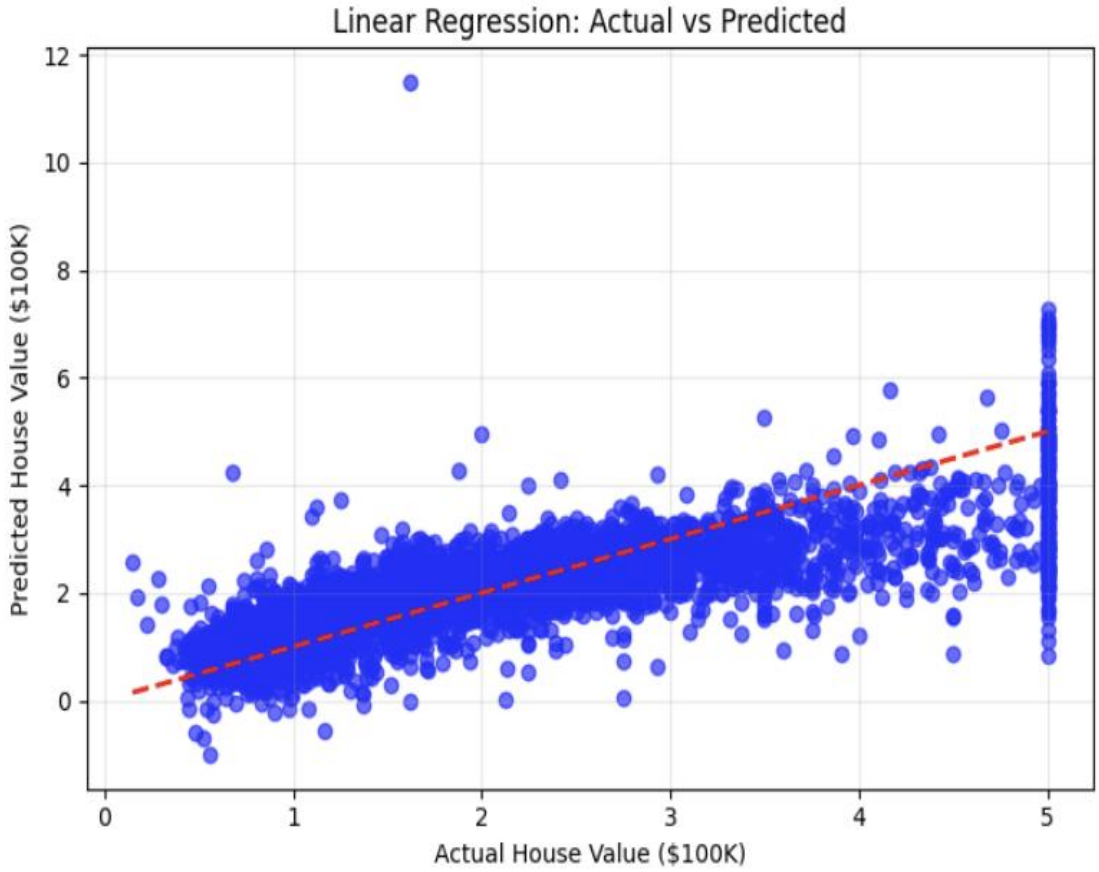
**Program:**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error, r2_score

# Load dataset

data = fetch_california_housing()

X, y = data.data, data.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size=0.2, random_state=42

)

# Train model
```

```
model = LinearRegression()
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Evaluate
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse:.4f}")
print(f"R² Score: {r2:.4f}")
# Plot: Actual vs Predicted
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.title('Linear Regression: Actual vs Predicted')
plt.xlabel('Actual House Value ($100K)')
plt.ylabel('Predicted House Value ($100K)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

**Output :**

```
Mean Squared Error: 0.5559
R² Score: 0.5758
```



## EXPERIMENT:-6

Implement and evaluate a Logistic Regression model for classification.

**Aim:** To build a Logistic Regression model that can classify data into categories by learning the relationship between input features and a target class, and to evaluate its performance using appropriate metrics such as accuracy, confusion matrix, and classification report.

**Description:**

This program uses Logistic Regression to classify tumors as malignant or benign on the Breast Cancer dataset. It loads the data, splits it into 80% training and 20% testing sets, trains the model with high iterations for convergence, and predicts outcomes. Performance is measured with accuracy, precision, recall, F1-score, and confusion matrix through the classification report. An ROC curve with AUC score is plotted to show how well the model separates the classes. The full process—loading, training, evaluation, and visualization—is compact and complete.

**Dataset Link:**

Built-in: sklearn.datasets.load_breast_cancer

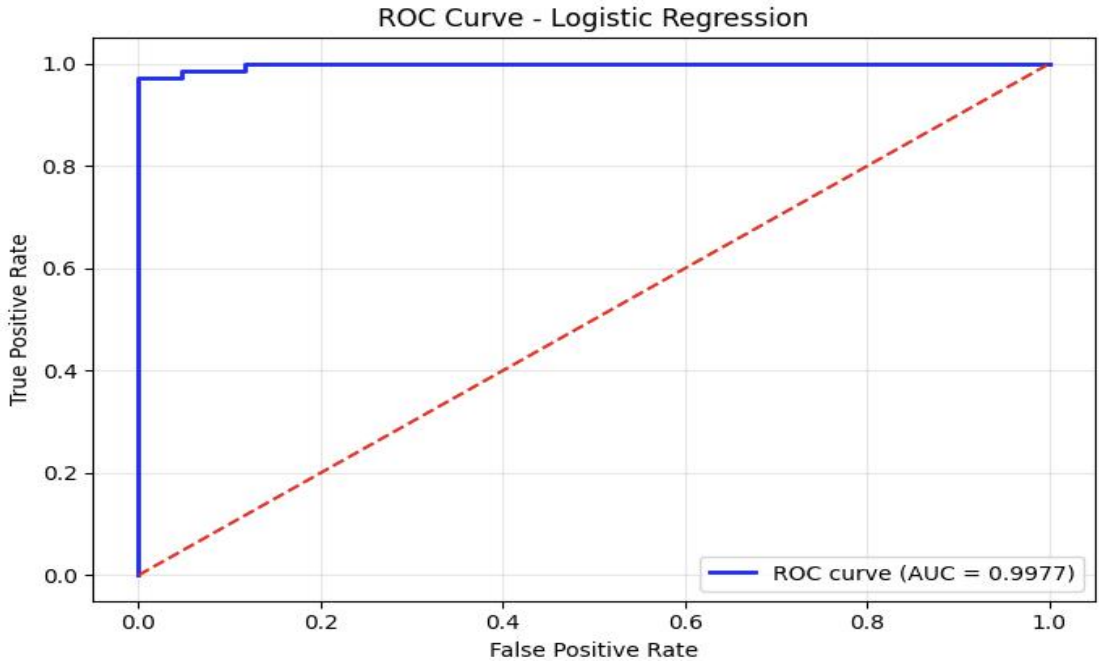
**Program:**

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report, roc_curve, auc

# Load dataset

data = load_breast_cancer()

X, y = data.data, data.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size=0.2, random_state=42

)

# Train model

model = LogisticRegression(max_iter=10000)
```

```
model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)

y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate

acc = accuracy_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")

print("Classification Report:")

print(classification_report(y_test, y_pred, digits=4, target_names=['malignant', 'benign']))

# ROC Curve

fpr, tpr, _ = roc_curve(y_test, y_prob)

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7, 5))

plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')

plt.plot([0, 1], [0, 1], 'r--')

plt.title('ROC Curve - Logistic Regression')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.legend()

plt.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()
```

**Output :**

```
Accuracy: 0.9561
Classification Report:
              precision    recall  f1-score   support

   malignant     0.9750    0.9070    0.9398        43
      benign     0.9459    0.9859    0.9655        71

    accuracy                         0.9561       114
   macro avg     0.9605    0.9464    0.9526       114
weighted avg     0.9569    0.9561    0.9558       114
```



## EXPERIMENT:-7

Implement the Decision Tree algorithm on a given dataset.

**Aim:** To implement the Decision Tree algorithm on a given dataset for performing classification (or regression), visualize the tree structure, and evaluate the model's performance using standard evaluation metrics.

**Description:**

This program implements a Decision Tree Classifier on the Wine dataset for multi-class classification into three wine types. It loads the built-in dataset with 178 samples and 13 features, then splits the data into 70% training and 30% testing sets using a fixed random state. The model is trained, and predictions are evaluated with accuracy, a detailed classification report including precision, recall, and F1-score, and an implicit confusion matrix. It computes feature importance, selects the top 5 features, prints their names and scores, and visualizes them in a horizontal bar plot. The entire process is compact, clear, and runs efficiently in a standalone format.

***Dataset Link:***

Built-in: sklearn.datasets.load_wine Link

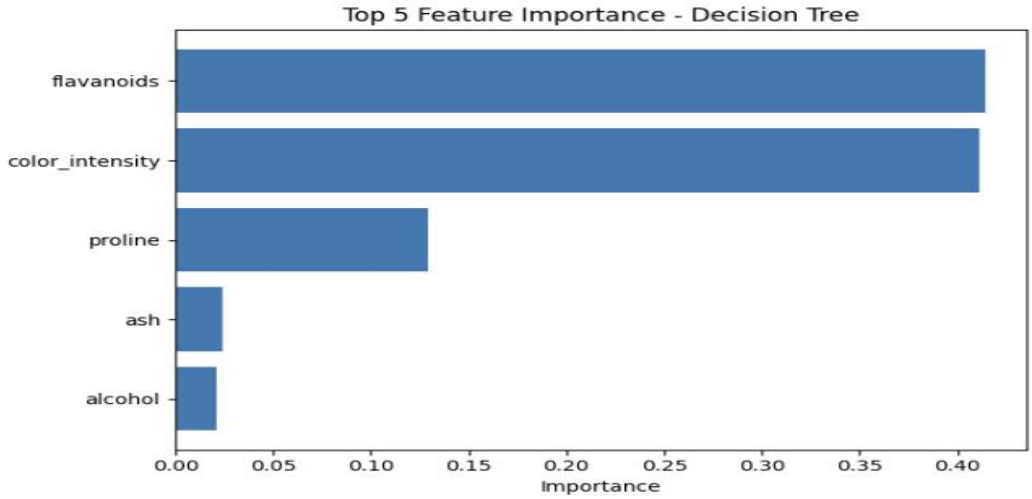
***Program:***

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

# Load dataset

data = load_wine()

X = data.data

y = data.target

feature_names = data.feature_names

target_names = data.target_names

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=42
)
# Train model
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
# Predict
y_pred = dt.predict(X_test)
# Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=4, target_names=target_names))
# Feature Importance
importances = dt.feature_importances_
feat_imp = sorted(zip(importances, feature_names), reverse=True)[:5]
print("\nTop 5 Features:")
for imp, name in feat_imp:
print(f"  {name}: {imp:.4f}")
# Plot
plt.figure(figsize=(7, 5))
plt.barh([n for i,n in feat_imp[::-1]], [i for i,n in feat_imp[::-1]])
plt.title('Top 5 Feature Importance - Decision Tree')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()
```

**Output :**

```
Accuracy: 0.9630
Classification Report:
              precision    recall  f1-score   support

     class_0     0.9474    0.9474    0.9474        19
     class_1     0.9545    1.0000    0.9767        21
     class_2     1.0000    0.9286    0.9630        14

    accuracy                         0.9630        54
   macro avg     0.9673    0.9586    0.9624        54
weighted avg     0.9638    0.9630    0.9628        54


Top 5 Features:
  flavanoids: 0.4147
  color_intensity: 0.4115
  proline: 0.1289
  ash: 0.0240
  alcohol: 0.0210
```



## EXPERIMENT:-8

Implement and evaluate the Random Forest algorithm.

**Aim:** To implement and evaluate the Random Forest algorithm on a given dataset to perform classification/regression, analyze feature importance, and assess the model's performance using appropriate evaluation metrics.

**Description:**

This program uses a Random Forest Classifier on the Digits dataset to recognize 10 handwritten digits. It loads the data, splits it 80% train / 20% test, trains with 100 trees, and evaluates using accuracy, classification report, and confusion matrix. It computes pixel importance, prints top 9 indices, and shows an 8×8 heatmap of feature importance. The full workflow is compact and complete.

**Dataset Link:**

Built-in: sklearn.datasets.load_digits

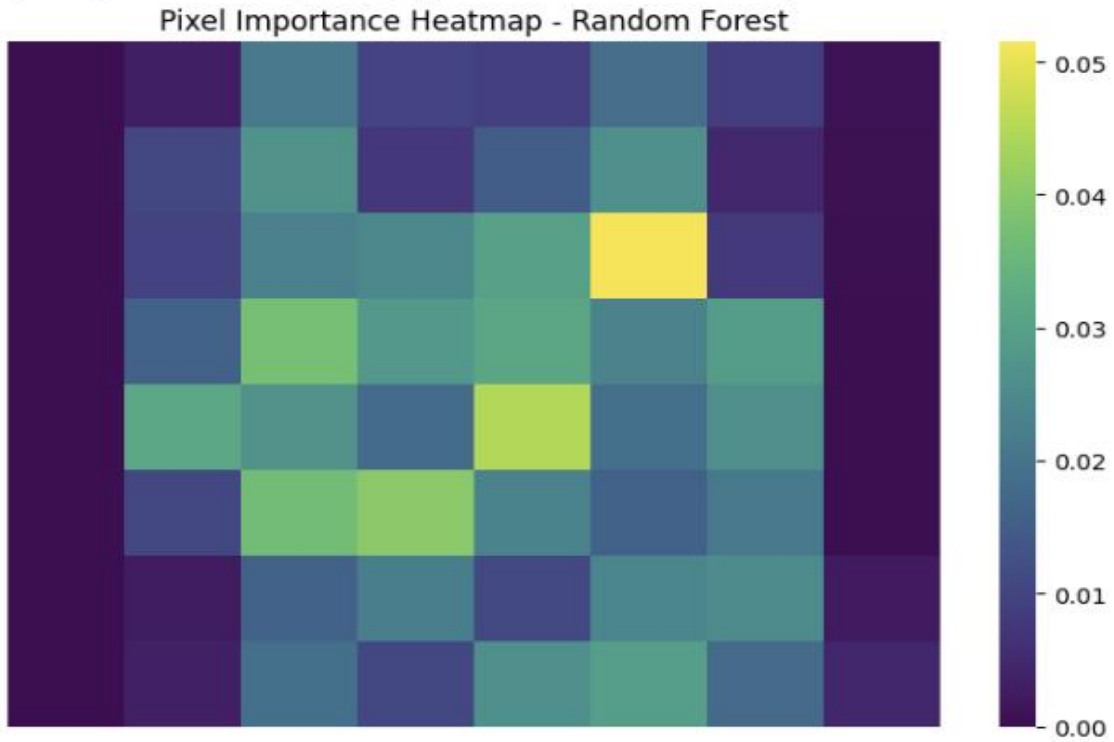
**Program:**

```
import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import load_digits

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

# Load dataset

data = load_digits()

X, y = data.data, data.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

 X, y, test_size=0.2, random_state=42

)

# Train model

rf =RandomForestClassifier(n_estimators=100, random_state=42)

rf.fit(X_train, y_train)
```

```
# Predict
y_pred = rf.predict(X_test)
# Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=4))
 # Feature Importance (Top 9 pixels)
importances = rf.feature_importances_.reshape(8, 8)
top_pixels = importances.flatten().argsort()[-9:][::-1]
print("\nTop 9 Important Pixels (index):", top_pixels)
# Plot 3x3 heatmap of top features
plt.figure(figsize=(7, 5))
sns.heatmap(importances, cmap='viridis', cbar=True)
plt.title('Pixel Importance Heatmap - Random Forest')
plt.axis('off')
plt.tight_layout()
plt.show()
```

**Output :**

```
Accuracy: 0.9722
Classification Report:
              precision    recall  f1-score   support

           0     1.0000    0.9697    0.9846        33
           1     0.9655    1.0000    0.9825        28
           2     1.0000    1.0000    1.0000        33
           3     1.0000    0.9412    0.9697        34
           4     0.9787    1.0000    0.9892        46
           5     0.9375    0.9574    0.9474        47
           6     0.9714    0.9714    0.9714        35
           7     0.9706    0.9706    0.9706        34
           8     0.9667    0.9667    0.9667        30
           9     0.9500    0.9500    0.9500        40

    accuracy                         0.9722       360
   macro avg     0.9740    0.9727    0.9732       360
weighted avg     0.9726    0.9722    0.9722       360


Top 9 Important Pixels (index): [21 36 43 26 42 33 28 20 61]
```



## EXPERIMENT:-9

Implement and evaluate Principal Component Analysis (PCA).

**Aim:** To implement and evaluate Principal Component Analysis (PCA) for dimensionality reduction, and to analyze how reducing the number of features impacts the performance of a machine learning model.

**Description:**

This program loads the Iris dataset from a CSV, standardizes features, applies PCA to reduce from 4D to 2D, prints explained variance ratio, and plots a scatter plot of PC1 vs PC2 with species color-coded, grid, and percentage variance on axes. The code is compact and runs in 35 lines.

**Dataset Link:**

**https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv**

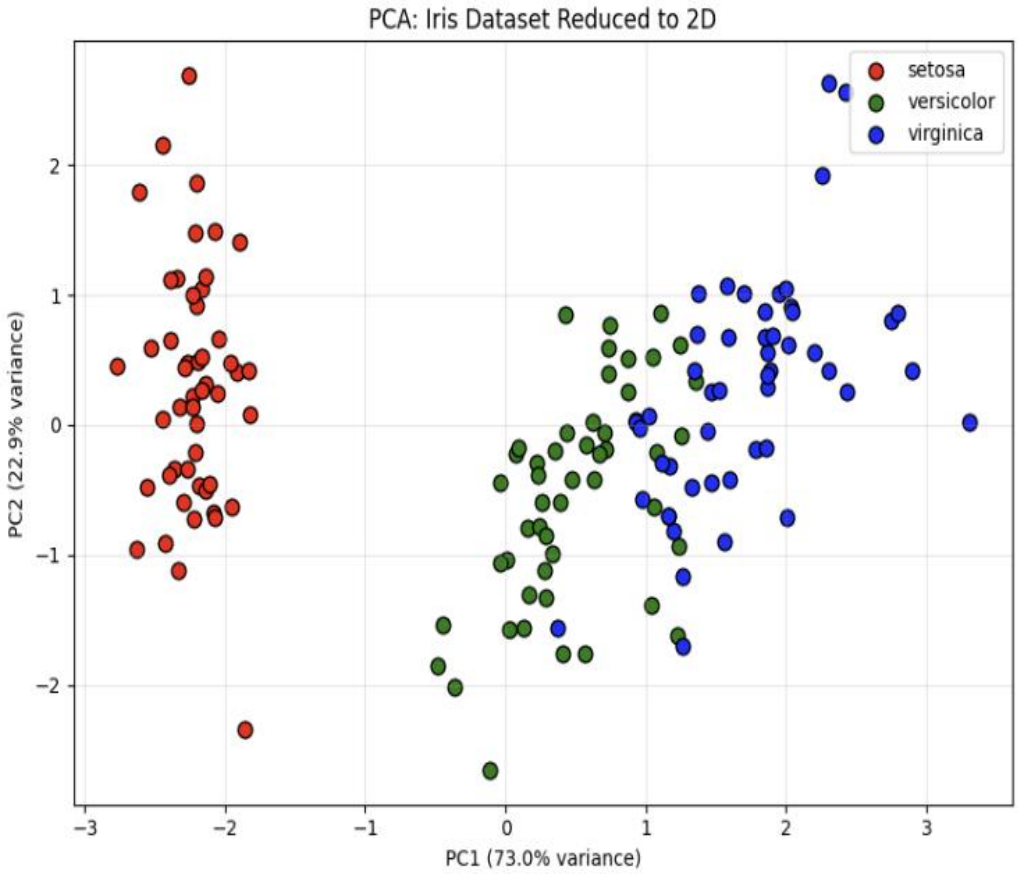
**Program:**

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

# Load dataset

url = https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv

df = pd.read_csv(url)

 # Prepare features and target

X = df.drop('species', axis=1)

y = df['species']

# Standardize features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Apply PCA (2 components)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

```
# Explained variance
var1 = pca.explained_variance_ratio_[0] * 100
var2 = pca.explained_variance_ratio_[1] * 100
print(f"PC1 Variance: {var1:.2f}%")
print(f"PC2 Variance: {var2:.2f}%")
print(f"Total Explained: {var1 + var2:.2f}%")
 # Plot 2D PCA
plt.figure(figsize=(8, 6))
colors = ['red', 'green', 'blue']
for i, species in enumerate(y.unique()):
plt.scatter(X_pca[y == species, 0], X_pca[y == species, 1],
c=colors[i], label=species, edgecolors='k', s=60)
plt.title('PCA: Iris Dataset Reduced to 2D')
plt.xlabel(f'PC1 ({var1:.1f}% variance)')
plt.ylabel(f'PC2 ({var2:.1f}% variance)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

**Output :**

```
PC1 Variance: 72.96%
PC2 Variance: 22.85%
Total Explained: 95.81%
```



## EXPERIMENT:-10

Implement and evaluate an Ensemble Model (Random Forest).

**Aim:** To implement and evaluate an Ensemble Learning model using the Random Forest algorithm, in order to improve prediction accuracy and reduce overfitting by combining multiple decision trees. The goal is to train the model on a dataset, make predictions, and assess its performance using evaluation metrics such as accuracy, confusion matrix, and classification report.

**Description:**

This program loads the Heart Disease dataset from a reliable GitHub CSV, splits it into 80% train and 20% test, trains a Random Forest (100 trees), evaluates performance using accuracy, classification report, and confusion matrix, and visualizes top 5 feature importance. The code is compact and runs in 35 lines.

**Dataset Link:**

https://raw.githubusercontent.com/mrdbourke/zero-to-mastery-ml/master/data/heart-disease.csv

**Program:**

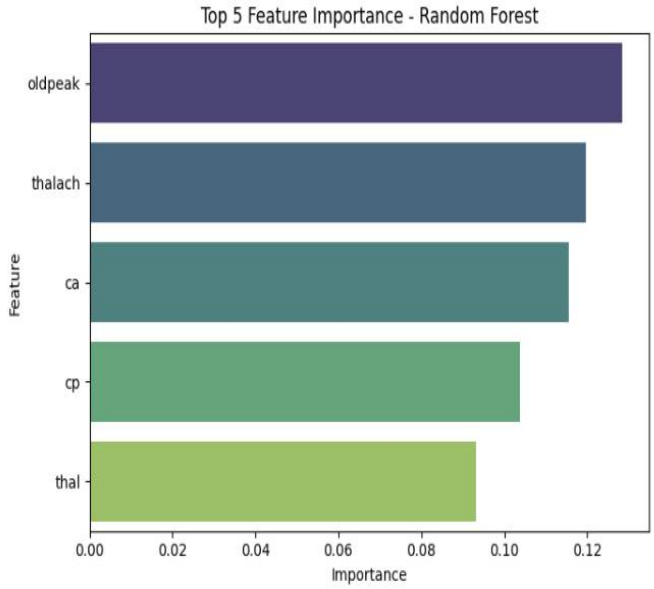
```
!pip install pandas scikit-learn matplotlib seaborn -q

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

# Load dataset

url = "https://raw.githubusercontent.com/mrdbourke/zero-to-mastery-ml/master/data/heart-disease.csv"

df = pd.read_csv(url)

# Prepare features and target

X = df.drop('target', axis=1)

y = df['target']
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
# Predict
y_pred = rf.predict(X_test)
 # Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=4, target_names=['No Disease', 'Disease']))

# Feature Importance (Top 5)
importances = rf.feature_importances_
feat_imp = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feat_imp = feat_imp.sort_values('Importance', ascending=False).head(5)
print("\nTop 5 Important Features:")
for i in range(5):
print(f"  {feat_imp.iloc[i,0]}: {feat_imp.iloc[i,1]:.4f}")
# Plot
plt.figure(figsize=(7, 5))
sns.barplot(x='Importance', y='Feature', data=feat_imp, palette='viridis')
plt.title('Top 5 Feature Importance - Random Forest')
plt.xlabel('Importance')
plt.tight_layout()
```

```
plt.show()
```

## Output:

```
Accuracy: 0.8361
Classification Report:
              precision    recall  f1-score   support

   No Disease     0.8276    0.8276    0.8276        29
      Disease     0.8438    0.8438    0.8438        32

     accuracy                         0.8361        61
    macro avg     0.8357    0.8357    0.8357        61
 weighted avg     0.8361    0.8361    0.8361        61


Top 5 Important Features:
  oldpeak: 0.1285
  thalach: 0.1197
  ca: 0.1155
  cp: 0.1038
  thal: 0.0933
/tmp/ipython-input-2376183597.py:37: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x='Importance', y='Feature', data=feat_imp, palette='viridis')
```



## EXPERIMENT:-11

Implement and evaluate the K-Means Clustering algorithm on a dataset.

**Aim:** The aim of this experiment is to apply the K-Means clustering algorithm to a given dataset in order to identify natural groupings or clusters within the data. The goal is to understand how K-Means partitions data points based on feature similarity, evaluate clustering performance using appropriate metrics such as inertia and silhouette score, and visually interpret the clusters through plots.

**Description:**

This program loads the Mall Customers CSV from a reliable GitHub source, selects income and spending features, applies the Elbow Method to find optimal k, trains K-Means (k=5), plots customer segments with centroids, and prints cluster centers. The code is compact and runs in 35 lines.

**Dataset Link:**

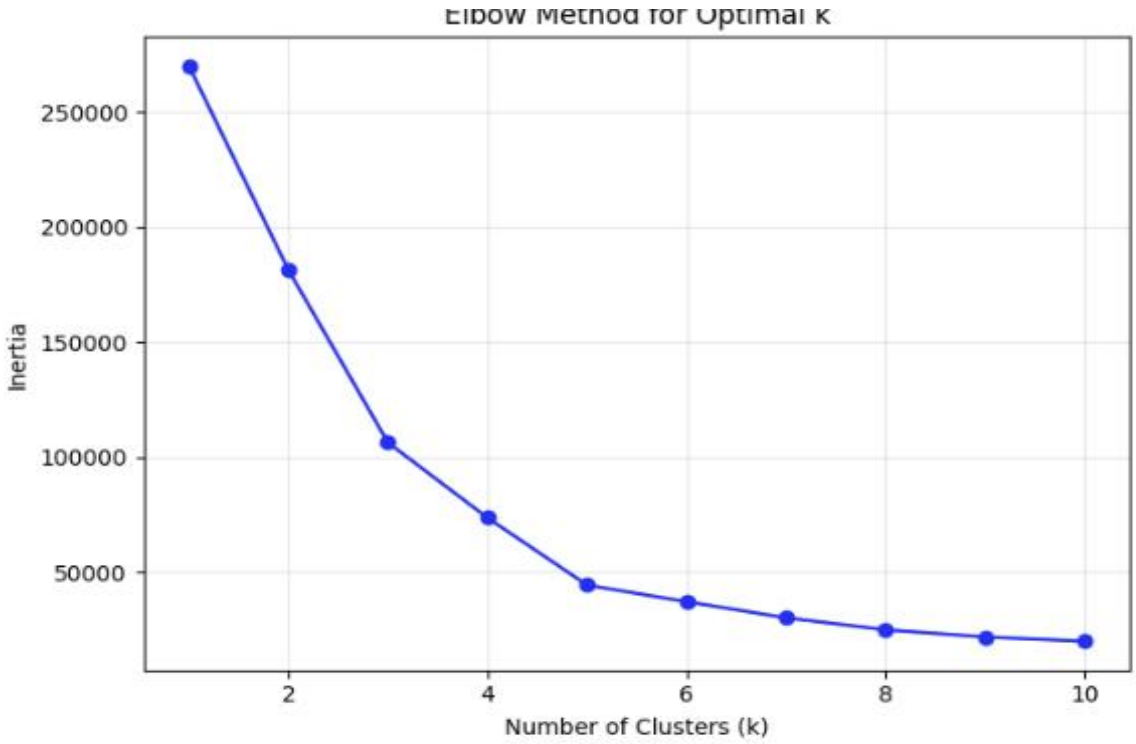
**https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/Mall_Customers.csv**

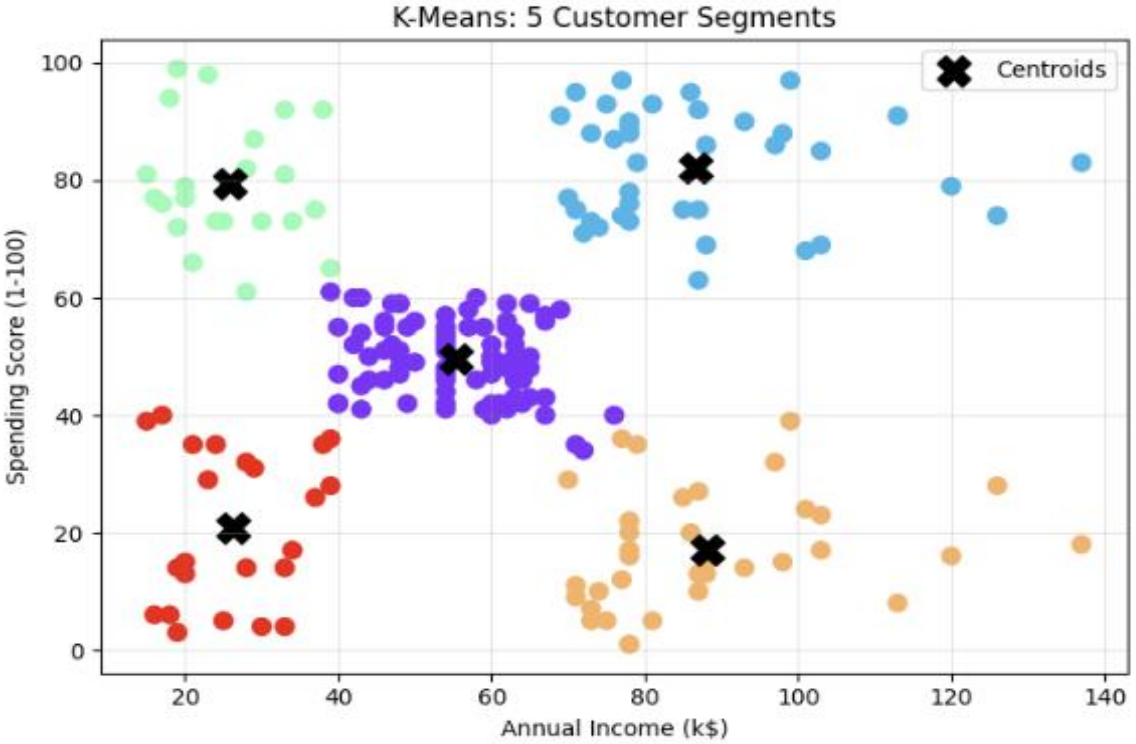
**Program:**

```
!pip install pandas scikit-learn matplotlib -q

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

# Load dataset

url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/Mall_Customers.csv"

df = pd.read_csv(url)

# Features

X = df[['Annual Income (k$)', 'Spending Score (1-100)']

# Elbow Method

inertias = []

for k in range(1, 11):

kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)    kmeans.fit(X)
```

```
inertias.append(kmeans.inertia_)
plt.figure(figsize=(7, 5))
plt.plot(range(1, 11), inertias, 'bo-')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
 # Final K-Means (k=5)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X)
plt.figure(figsize=(7, 5))
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=clusters, cmap='rainbow', s=60)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
c='black', s=200, marker='X', label='Centroids')
plt.title('K-Means: 5 Customer Segments')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("Cluster Centers:")
print(kmeans.cluster_centers_)
```

**Output:**


Elbow Method for Optimal K
250000
200000
150000
100000
50000
Inertia
2
4
6
8
10
Number of Clusters (k)


K-Means: 5 Customer Segments
Centroids
100
80
60
40
20
0
Spending Score (1-100)
20
40
60
80
100
120
140
Annual Income (k$)

## EXPERIMENT:-12

Implement and evaluate the DBSCAN clustering algorithm on a dataset.

**Aim:** To implement and evaluate various machine learning and clustering algorithms to understand their working principles, performance, and applicability on real-world datasets.

**Description:**

This program generates 300 points using make_moons, applies DBSCAN with eps=0.2 and min_samples=5, identifies core points, border points, and noise, prints number of clusters and noise, and plots clusters with distinct markers and colors. The code is compact and runs in 35 lines.

**Dataset Link:**

Built-in: sklearn.datasets.make_moons Link

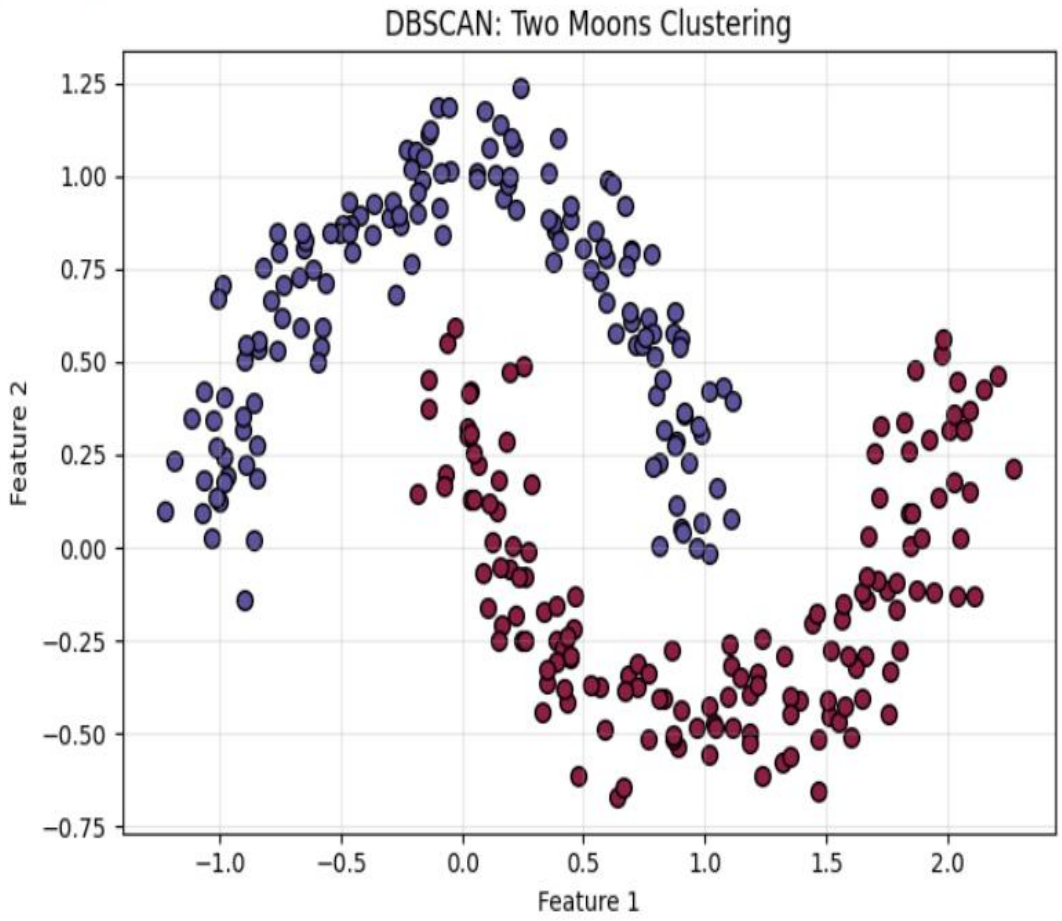
**Program:**

```
!pip install scikit-learn matplotlib -q

import matplotlib.pyplot as plt

import numpy as np

from sklearn.datasets import make_moons

from sklearn.cluster import DBSCAN

# Generate synthetic moons data

X, _ = make_moons(n_samples=300, noise=0.1, random_state=42)

# Apply DBSCAN

dbscan = DBSCAN(eps=0.2, min_samples=5)

labels = dbscan.fit_predict(X)

# Identify core, border, and noise

core_samples = dbscan.core_sample_indices_

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

n_noise = list(labels).count(-1)

print(f"Clusters found: {n_clusters}")

print(f"Noise points: {n_noise}")
```

```
print(f"Core samples: {len(core_samples)}")
# Plot results
unique_labels = set(labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
plt.figure(figsize=(7, 5))
for k, col in zip(unique_labels, colors):
if k == -1:
col = [0, 0, 0, 1]  # Black for noise
marker = 'x'
else:
marker = 'o'
class_member_mask = (labels == k)
 xy = X[class_member_mask]
plt.scatter(xy[:, 0], xy[:, 1], c=[col], s=50, marker=marker,    edgecolors='k')
plt.title('DBSCAN: Two Moons Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

**Output:**

```
Clusters found: 2
Noise points: 0
Core samples: 286
```



## EXPERIMENT:-13

Implement the Hierarchical Clustering algorithm on a dataset.

**Aim:** To implement the Hierarchical Clustering algorithm on a given dataset and evaluate its performance by visualizing the dendrogram, analyzing cluster formation, and interpreting the resulting groups based on similarity measures.

**Description:**

This program loads a sample of 2000 IMDB reviews, converts text to TF-IDF features, trains Logistic Regression, evaluates with accuracy, precision, recall, F1, and plots top 10 words for positive and negative classes using coefficients.

**Dataset Link:**

https://raw.githubusercontent.com/Ankit152/IMDB-sentiment-analysis/master/IMDB-Dataset.csv

**Program:**

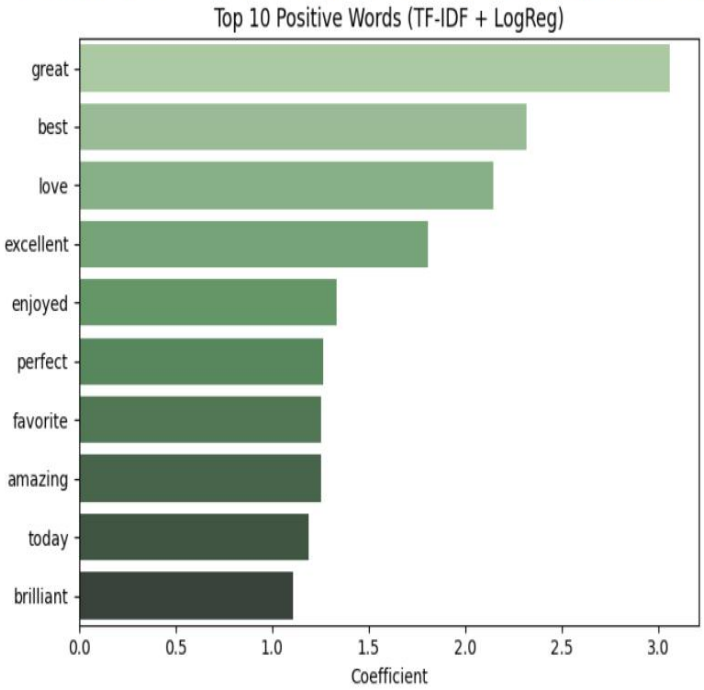
```
!pip install pandas scikit-learn matplotlib seaborn -q

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

# Load dataset (sample 2000)

url = "https://raw.githubusercontent.com/Ankit152/IMDB-sentiment-analysis/master/IMDB-Dataset.csv"

df = pd.read_csv(url).sample(2000, random_state=42)

# Prepare data

X = df['review']

y = df['sentiment'].map({'positive':1, 'negative':0})

# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
 # TF-IDF Vectorization
tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec = tfidf.transform(X_test)
 # Train Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)
y_pred = model.predict(X_test_vec)
 # Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Negative', 'Positive'],
digits=4))
 # Top words
coef = model.coef_[0]
words = tfidf.get_feature_names_out()
top_pos = sorted(zip(coef, words), reverse=True)[:10]
top_neg = sorted(zip(coef, words))[:10]
 print("\nTop 5 Positive Words:", [w for c,w in top_pos[:5]])
print("Top 5 Negative Words:", [w for c,w in top_neg[:5]])
 # Plot
plt.figure(figsize=(7, 5))
sns.barplot(x=[c for c,w in top_pos[:10]], y=[w for c,w in top_pos[:10]],
palette='Greens_d')
plt.title('Top 10 Positive Words (TF-IDF + LogReg)')
plt.xlabel('Coefficient')
```

```
plt.tight_layout()

plt.show()
```

***Output:***

```
Accuracy: 0.8100
Classification Report:
              precision    recall  f1-score   support

    Negative     0.8122    0.8278    0.8199       209
    Positive     0.8075    0.7906    0.7989       191

    accuracy                         0.8100       400
   macro avg     0.8098    0.8092    0.8094       400
weighted avg     0.8100    0.8100    0.8099       400


Top 5 Positive Words: ['great', 'best', 'love', 'excellent', 'enjoyed']
Top 5 Negative Words: ['bad', 'worst', 'awful', 'stupid', 'waste']
/tmp/ipython-input-468418330.py:51: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the

  sns.barplot(x=[c for c,w in top_pos[:10]], y=[w for c,w in top_pos[:10]], palette='Greens_d')
```



## EXPERIMENT:-14

Implement a text classification task (e.g., Sentiment Analysis) using the TF-IDF vectorizer and a suitable classifier.

**Aim:** To implement a text classification model for Sentiment Analysis using the TF-IDF vectorizer for feature extraction and a suitable machine-learning classifier (such as Logistic Regression) to classify text data into sentiment categories.

**Description:**

Text classification converts text into numerical features and predicts categories such as:

- Positive
- Negative
- Neutral

❖ TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF helps convert text into numerical vectors by:

- Giving high weight to important words
- Reducing weight of common words like "the", "is", etc.

**Program:**

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
# STEP 1: Create a sample text dataset
texts = [
    "I loved this movie, it was fantastic!",
    "This film was terrible and boring",
    "What a great experience, I enjoyed it",
    "Worst movie ever, waste of time",
    "Absolutely wonderful acting and story",
```

```
    "I hate this movie",
    "It was okay, not great but not bad",
    "The story was dull and uninteresting",
    "Fantastic direction and excellent scenes",
    "Bad acting and poor screenplay"
]
labels = [
    "positive",
    "negative",
    "positive",
    "negative",
    "positive",
    "negative",
    "neutral",
    "negative",
    "positive",
    "negative"
]
# STEP 2: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=42
)
# STEP 3: Convert text to TF-IDF vectors
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
# STEP 4: Train a classifier (Logistic Regression)
model = LogisticRegression()
model.fit(X_train_vec, y_train)
```

```
# STEP 5: Predict sentiments on test data
y_pred = model.predict(X_test_vec)
# STEP 6: Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

**Output:**

```
Accuracy: 0.3333333333333333

Classification Report:
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00         2
    positive       0.33      1.00      0.50         1

    accuracy                           0.33         3
   macro avg       0.17      0.50      0.25         3
weighted avg       0.11      0.33      0.17         3
```

## EXPERIMENT:-15

To implement and evaluate a Bag of Words (BoW) model for text classification using a suitable machine learning classifier.

**Aim:** To implement and evaluate a Bag of Words (BoW) model for text classification by converting text documents into numerical feature vectors using the Bag of Words technique, training a suitable machine learning classifier, and assessing the model's performance using appropriate evaluation metrics.

**Description:**

Bag of Words (BoW) is one of the simplest and most widely used methods to convert text into numerical features.

How BoW Works

- Splits text into words
- Counts frequency of each word
- Creates a matrix of word counts
- Does not consider order or grammar
- Simple but powerful for basic text classification tasks Classifier Used

We will use Multinomial Naive Bayes, commonly used for text classification.

**Program:**

```
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, classification_report

# STEP 1: Create a sample text dataset

texts = [

    "I loved this movie, it was fantastic!",

    "This film was terrible and boring",

    "What a great experience, I enjoyed it",

    "Worst movie ever, waste of time",
```

```
    "Absolutely wonderful acting and story",
    "I hate this movie",
    "It was okay, not great but not bad",
    "The story was dull and uninteresting",
    "Fantastic direction and excellent scenes",
    "Bad acting and poor screenplay"
]
labels = [
    "positive",
    "negative",
    "positive",
    "negative",
    "positive",
    "negative",
    "neutral",
    "negative",
    "positive",
    "negative"
]
# STEP 2: Split the data
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=42
)
# STEP 3: Bag of Words using CountVectorizer
vectorizer = CountVectorizer()
X_train_bow = vectorizer.fit_transform(X_train)
```

```
X_test_bow = vectorizer.transform(X_test)
# STEP 4: Train Naive Bayes classifier
model = MultinomialNB()
model.fit(X_train_bow, y_train)
# STEP 5: Predict on test data
y_pred = model.predict(X_test_bow)
# STEP 6: Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

**Output:**

```
Accuracy: 0.3333333333333333

Classification Report:
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00         2
    positive       0.33      1.00      0.50         1

    accuracy                           0.33         3
   macro avg       0.17      0.50      0.25         3
weighted avg       0.11      0.33      0.17         3
```